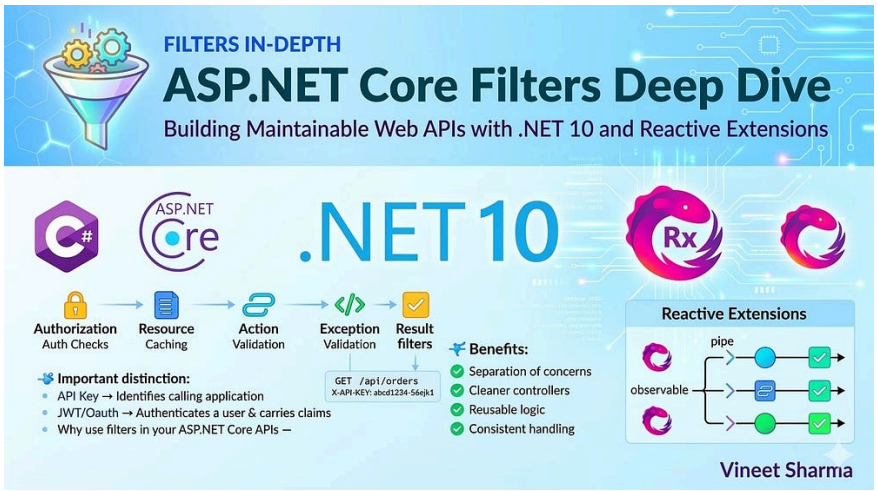
ASP.NET Core Filters Deep Dive: Building Maintainable Web APIs with .NET 10 and Reactive Extensions

Build production-ready Web APIs with proper cross-cutting concern separation using the latest .NET 10 features



ASP.NET Core Filters Deep Dive: Building Maintainable Web APIs with .NET 10 and Reactive Extensions

The Structured Way to Handle Cross-Cutting Concerns for Cleaner, More Reactive APIs

In the world of software architecture, “cross-cutting concerns” are the silent killers of clean code. Logging, authentication, validation, and exception handling—these tasks are essential, but they often clutter your core business logic, leading to bloated controllers and a maintenance nightmare.

For years, ASP.NET Core has provided an elegant solution to this problem: Filters. They allow you to intercept and modify the request-processing pipeline, keeping your controllers lean and focused.

With the release of .NET 10, the filter architecture has become even more powerful and flexible, now with first-class support for Minimal

APIs. In this story, we'll explore each filter type in detail while demonstrating four key benefits:

- ✅ Separation of Concerns
- ✅ Cleaner Controllers/Endpoints
- ✅ Reusable Logic across Endpoints
- ✅ Consistent Logging and Error Handling

We'll also break away from traditional imperative examples and embrace Reactive Programming principles using `System.Reactive`, showing you how to build APIs that are not just clean, but also event-driven and resilient.

Document Structure

The Filter Pipeline

- Execution Order Overview

1. Authorization Filters

- API Key + Role-Based Authorization
- Controller & Minimal API Examples

2. Resource Filters

- Reactive Performance Monitoring
- Real-Time Alerting with Background Services

3. Action Filters

- Reactive Validation & Audit Logging
- Clean Controllers with Zero Validation Code

4. Exception Filters

- Global Error Handling
- Error Monitoring with Real-Time Alerts

5. Result Filters

- Standardized API Responses
- Caching Headers & Response Envelopes

Putting It All Together

- Complete `Program.cs` Configuration
- Summary: The Four Benefits Achieved

.NET 10 vs Previous Versions

- Key Improvements in Filter Registration

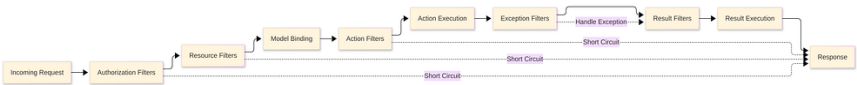
Reactive Streams Architecture

- How Filters Feed Decoupled Monitoring

. . .

The Filter Pipeline: A Visual Overview

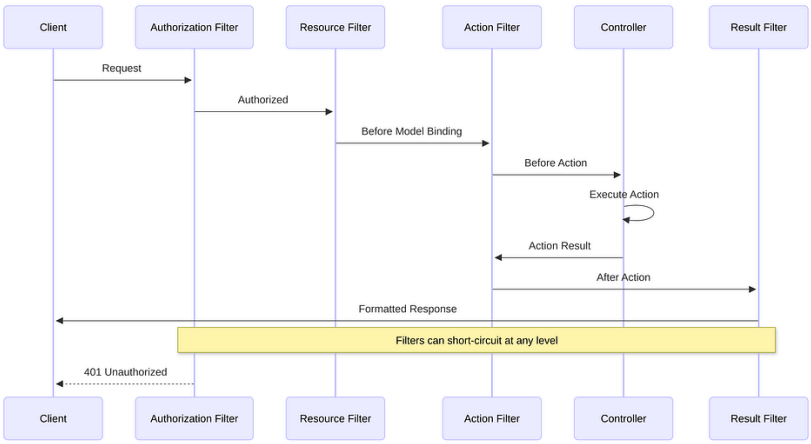
Before diving into code, let's visualize the order in which filters execute. Think of it as a Russian doll, where each filter wraps the next.



The Filter Pipeline: A Visual Overview [View Source]

This structured pipeline is your toolkit for building maintainable APIs.

Filter Execution Order with Multiple Filters



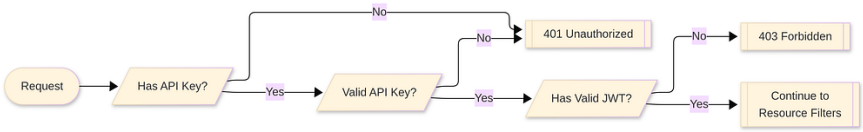
Filter Execution Order with Multiple Filters [View Source]

• • •

1. Authorization Filters: The Gatekeepers

What They Do: Authorization Filters run first in the pipeline. Their sole purpose is to determine if the current request is authorized. If a filter determines the request is not authorized, it short-circuits the pipeline, returning a 401 (Unauthorized) or 403 (Forbidden) response immediately.

Authorization Flow



Authorization Flow [View Source]

The Four Benefits in Action:

- **Separation of Concerns:** Authentication logic is extracted from business logic
- **Cleaner Endpoints:** Controllers don't need to check API keys or roles
- **Reusable Logic:** Same filter can protect multiple endpoints

- **Consistent Security:** Every protected endpoint follows the same rules

Code Example: API Key + Role-Based Authorization

Let's create a custom filter that checks for an API key (identifying the application) and then, based on that application, ensures a user has a specific role (from a JWT).

```
using Microsoft.AspNetCore.Mvc.Filters;
using Microsoft.AspNetCore.Mvc;
using System.Reactive.Linq;
using System.Security.Claims;

// ==============================================
// FILTER IMPLEMENTATION - REUSABLE COMPONENT
// ==============================================
public class ApiKeyAndRoleFilter : IAuthorizatic
{
    private readonly string _role;
    private const string ApiKeyHeaderName = "X-A
    public ApiKeyAndRoleFilter(string role)
    {
        _role = role;
    }
    public void OnAuthorization(AuthorizationFil
    {
        // Check for API Key (identifies the app
        if (!context.HttpContext.Request.Headers
        {
            // Consistent error response format
            context.Result = new UnauthorizedObj
            {
                Error = "API Key missing",
                Timestamp = DateTime.UtcNow,
                RequestId = context.HttpContext.
            });
            return;
        }
        // Validate the API Key (in real apps, t
        var isValidApiKey = IsValidApiKey(extrac

        if (!isValidApiKey)
```

```
            {
                context.Result = new UnauthorizedObj
                {
                    Error = "Invalid API Key",
                    Timestamp = DateTime.UtcNow,
                    RequestId = context.HttpContext.
                });
                return;
            }
            // Check user's role from JWT (authentic
            var user = context.HttpContext.User;
            if (!user.Identity?.IsAuthenticated ?? f
            {
                context.Result = new ForbidResult("U
                return;
            }
        }
        private bool IsValidApiKey(string apiKey)
        {
            // In production, this would validate ag
            return apiKey == "abcd1234-56ejk1" || ap
        }
    }
    // ==============================================
    // CONTROLLER-BASED API - CLEAN AND FOCUSED
    // ==============================================
    [ApiController]
    [Route("api/[controller]")]
    public class OrdersController : ControllerBase
    {
        // NO AUTHENTICATION CODE HERE!
        // Just pure business logic - Separation of

        [HttpGet]
        [ServiceFilter(typeof(ApiKeyAndRoleFilter))]
        public IActionResult GetOrders()
        {
            // Clean controller - only business logi
            var orders = new[]
            {
                new { Id = 1, Product = "Laptop", St
                new { Id = 2, Product = "Mouse", Sta
            };

            // Consistent response format
            return Ok(new
```

```
            {
                Data = orders,
                Count = orders.Length,
                Timestamp = DateTime.UtcNow
            });
        }

        [HttpGet("{id}")]
        [ServiceFilter(typeof(ApiKeyAndRoleFilter))]
        public IActionResult GetOrder(int id)
        {
            // Still clean, still focused
            var order = new { Id = id, Product = "La
            return Ok(order);
        }
    }
    // ==============================================
    // MINIMAL API APPROACH (.NET 10+)
    // ==============================================
    // In Program.cs or a separate endpoint definiti
    public static class OrderEndpoints
    {
        public static void MapOrderEndpoints(this IE
        {
            // Group with common configuration
            var orderGroup = app.MapGroup("/api/mini
                .AddEndpointFilter<ApiKeyAndRoleFilt

            // Clean, focused endpoints
            orderGroup.MapGet("/", () =>
            {
                // No auth code here either!
                var orders = new[]
                {
                    new { Id = 1, Product = "Laptop"
                    new { Id = 2, Product = "Mouse",
                };

                return Results.Ok(new
                {
                    Data = orders,
                    Count = orders.Length,
                    Timestamp = DateTime.UtcNow
                });
            });
```

```
            orderGroup.MapGet("/{id}", (int id) =>
            {
                var order = new { Id = id, Product =
                return Results.Ok(order);
            });
        }
    }
```

What's New in .NET 10?

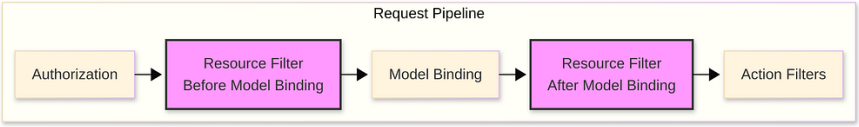
The `.AddEndpointFilter<TApiKeyAndRoleFilter>()` syntax brings the same filter pattern to Minimal APIs that controller developers have enjoyed for years. In previous versions (.NET 6-9), you had to implement `IEndpointFilter` separately, creating inconsistency between API styles.

. . .

2. Resource Filters: The Caching & Monitoring Interceptors

What They Do: Resource Filters execute *before* and *after* model binding. This is the perfect place for operations that need to inspect or modify the raw request/response, such as caching or performance monitoring.

Resource Filter Pipeline Position



Resource Filter Pipeline Position [View Source]

The Four Benefits in Action:

- **Separation of Concerns:** Performance monitoring is extracted from business logic

- **Cleaner Endpoints:** Controllers don't contain timing code
- **Reusable Logic:** Same monitoring logic across all endpoints
- **Consistent Metrics:** Every request is measured the same way

Code Example: Reactive Performance Monitoring

```
using Microsoft.AspNetCore.Mvc.Filters;
using Microsoft.AspNetCore.Mvc.Filters;
using System.Diagnostics;
using System.Reactive.Subjects;

// ==============================================
// REACTIVE STREAM FOR DECOUPLED MONITORING
// ==============================================
public static class PerformanceMetrics
{
    // Hot observable for real-time monitoring
    public static Subject<(string Path, long Ela

    // Derived streams for different metrics
    public static IObservable<double> AverageRes
        RequestTimings
            .Buffer(TimeSpan.FromMinutes(5))
            .Select(batch => batch.Average(t =>
            .Replay(1)
            .RefCount();
}
// ==============================================
// RESOURCE FILTER - REUSABLE MONITORING COMPONE
// ==============================================
public class ReactivePerformanceMonitorFilter :
{
    private readonly Stopwatch _stopwatch;
    private readonly ILogger<ReactivePerformance
    public ReactivePerformanceMonitorFilter(ILog
    {
        _stopwatch = new Stopwatch();
        _logger = logger;
    }
    public void OnResourceExecuting(ResourceExec
    {
```

```
            _stopwatch.Start();

            // Add correlation ID for tracking acros
            var correlationId = Guid.NewGuid().ToStr
            context.HttpContext.Items["CorrelationId
            context.HttpContext.Response.Headers["X-
        }
        public void OnResourceExecuted(ResourceExecu
        {
            _stopwatch.Stop();

            var path = context.HttpContext.Request.P
            var statusCode = context.HttpContext.Res
            var correlationId = context.HttpContext.

            // Push metrics to reactive stream
            PerformanceMetrics.RequestTimings.OnNext

            // Log for immediate visibility
            _logger.LogInformation(
                "Request {Path} completed with {Stat
                path, statusCode, _stopwatch.Elapsed
        }
        // Async version for non-blocking operations
        public async Task OnResourceExecutionAsync(R
        {
            OnResourceExecuting(context);

            try
            {
                await next();
            }
            finally
            {
                OnResourceExecuted(await CreateExecu
            }
        }

        private Task<ResourceExecutedContext> Create
        {
            // Helper to convert contexts (simplifie
            return Task.FromResult(new ResourceExecu
        }
    }
    // ==============================================
    // BACKGROUND SERVICE FOR REACTIVE PROCESSING
```

```
// ==============================================
public class PerformanceAlertingService : Backgr
{
    private readonly ILogger<PerformanceAlerting
    public PerformanceAlertingService(ILogger<Pe
    {
        _logger = logger;
    }
    protected override async Task ExecuteAsync(C
    {
        // Subscribe to slow requests (> 1000ms)
        PerformanceMetrics.RequestTimings
            .Where(t => t.ElapsedMs > 1000)
            .Subscribe(timing =>
            {
                _logger.LogWarning(
                    "SLOW REQUEST DETECTED: {Pat
                    timing.Path, timing.ElapsedM
            });
        // Monitor error rate
        PerformanceMetrics.RequestTimings
            .Buffer(TimeSpan.FromMinutes(1))
            .Subscribe(batch =>
            {
                var errorCount = batch.Count(t =
                if (errorCount > 10)
                {
                    _logger.LogCritical("High er
                }
            });
        // Keep the service running
        while (!stoppingToken.IsCancellationRequ
        {
            await Task.Delay(1000, stoppingToken
        }
    }
}
// ==============================================
// CONTROLLER USAGE
// ==============================================
[ApiController]
[Route("api/[controller]")]
[ServiceFilter(typeof(ReactivePerformanceMonitor
public class ProductsController : ControllerBase
{
    [HttpGet]
```

```
        public IActionResult GetProducts()
        {
            // Pure business logic - no performance
            var products = new[]
            {
                new { Id = 1, Name = "Laptop", Price
                new { Id = 2, Name = "Mouse", Price
            };

            // Simulate some work
            Thread.Sleep(100); // In real apps, this

            return Ok(products);
        }

        [HttpGet("slow")]
        public IActionResult GetSlowReport()
        {
            // Even this "slow" endpoint gets automa
            Thread.Sleep(2000); // Simulate slow ope
            return Ok(new { Report = "Monthly Sales"
        }
    }
    // ==============================================
    // MINIMAL API USAGE (.NET 10)
    // ==============================================
    var builder = WebApplication.CreateBuilder();
    builder.Services.AddSingleton<PerformanceAlertin
    builder.Services.AddScoped<ReactivePerformanceMo
    var app = builder.Build();
    // Apply filter to specific group
    var apiGroup = app.MapGroup("/api/v1")
        .AddEndpointFilter<ReactivePerformanceMonito
    apiGroup.MapGet("/products", () =>
    {
        var products = new[] { new { Id = 1, Name =
        return Results.Ok(products);
    });
    apiGroup.MapGet("/categories", () =>
    {
        var categories = new[] { new { Id = 1, Name
        return Results.Ok(categories);
    });
    // Start the alerting service
```

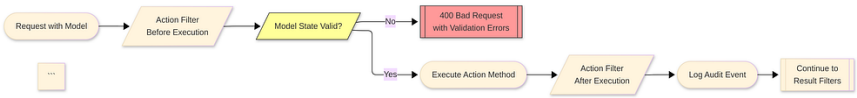
```
app.Services.GetRequiredService<PerformanceAlert
app.Run();
```

. . .

3. Action Filters: The Workhorses (Logging & Validation)

What They Do: Action Filters wrap the execution of your controller action. You can modify the arguments passed to the action or examine the result it returns.

Action Filter Flow with Validation



Action Filter Flow with Validation {View Source}

The Four Benefits in Action:

- **Separation of Concerns:** Input validation is separate from business logic
- **Cleaner Endpoints:** No validation clutter in action methods
- **Reusable Logic:** Same validation rules across different endpoints
- **Consistent Responses:** Uniform error format for validation failures

Code Example: Reactive Validation and Audit Logging

```
using Microsoft.AspNetCore.Mvc.Filters;
using System.ComponentModel.DataAnnotations;
using System.Reactive.Subjects;
```

```
using System.Text.Json;

// ==============================================
// AUDIT STREAM FOR DECOUPLED LOGGING
// ==============================================
public static class AuditLogStream
{
    public static Subject<AuditEvent> AuditEvent
}
public record AuditEvent(
    string Action,
    string User,
    object Arguments,
    object Result,
    DateTime Timestamp,
    string IpAddress,
    string CorrelationId);
// ==============================================
// CUSTOM VALIDATION ATTRIBUTE
// ==============================================
public class ValidOrderAttribute : ValidationAtt
{
    protected override ValidationResult? IsValid
    {
        if (value is OrderDto order)
        {
            if (order.Quantity <= 0)
            {
                return new ValidationResult("Qua
            }

            if (order.Price <= 0)
            {
                return new ValidationResult("Pri
            }

            if (string.IsNullOrWhiteSpace(order.
            {
                return new ValidationResult("Pro
            }
        }

        return ValidationResult.Success;
    }
}
// ==============================================
```

```
// ACTION FILTER FOR VALIDATION & AUDIT
// ============================================
public class ReactiveValidationAndAuditFilter :
{
    private readonly ILogger<ReactiveValidationA
    public ReactiveValidationAndAuditFilter(ILog
    {
        _logger = logger;
    }
    public async Task OnActionExecutionAsync(Act
    {
        // ===== BEFORE ACTION EXECUTION =====
        // Validate model state
        if (!context.ModelState.IsValid)
        {
            var errors = context.ModelState
                .Where(e => e.Value?.Errors.Coun
                .ToDictionary(
                    e => e.Key,
                    e => e.Value?.Errors.Select(
                );

            // Consistent validation error respo
            context.Result = new BadRequestObjec
            {
                Error = "Validation failed",
                Details = errors,
                Timestamp = DateTime.UtcNow,
                CorrelationId = context.HttpCont
            });

            // Log validation failure to audit s
            AuditLogStream.AuditEvents.OnNext(ne
                Action: context.ActionDescriptor
                User: context.HttpContext.User.I
                Arguments: context.ActionArgumen
                Result: new { Error = "Validatio
                Timestamp: DateTime.UtcNow,
                IpAddress: context.HttpContext.C
                CorrelationId: context.HttpConte
            ));

            return;
        }

        // ===== EXECUTE ACTION =====
```

```
            var executedContext = await next();

            // ===== AFTER ACTION EXECUTION =====
            // Audit successful execution
            if (executedContext.Exception == null)
            {
                AuditLogStream.AuditEvents.OnNext(ne
                    Action: context.ActionDescriptor
                    User: context.HttpContext.User.I
                    Arguments: context.ActionArgumen
                    Result: executedContext.Result!,
                    Timestamp: DateTime.UtcNow,
                    IpAddress: context.HttpContext.C
                    CorrelationId: context.HttpConte
                ));

                _logger.LogInformation(
                    "Action {Action} completed succe
                    context.ActionDescriptor.Display
                    context.HttpContext.User.Identit
            }
        }
    }
    // ==============================================
    // DTO WITH VALIDATION
    // ==============================================
    public record OrderDto
    {
        [Required]
        [StringLength(100)]
        public string ProductName { get; init; } = s

        [Range(1, 1000)]
        public int Quantity { get; init; }

        [Range(0.01, 10000)]
        public decimal Price { get; init; }

        [EmailAddress]
        public string? CustomerEmail { get; init; }
    }
    // ==============================================
    // CONTROLLER WITH CLEAN ACTIONS
    // ==============================================
    [ApiController]
    [Route("api/[controller]")]
```

```
[ServiceFilter(typeof(ReactiveValidationAndAudit
public class OrdersV2Controller : ControllerBase
{
    private static readonly List<OrderDto> _orde

    [HttpPost]
    public IActionResult CreateOrder([FromBody][
    {
        // NO VALIDATION CODE HERE!
        // NO LOGGING CODE HERE!
        // Just pure business logic

        _orders.Add(order);

        return Created($"/api/orders/{_orders.Co
        {
            Message = "Order created successfull
            OrderId = _orders.Count,
            Order = order
        });
    }

    [HttpPut("{id}")]
    public IActionResult UpdateOrder(int id, [Fr
    {
        // Still clean - validation handled by f
        if (id <= 0 || id > _orders.Count)
        {
            return NotFound(new { Error = "Order
        }

        _orders[id - 1] = order;

        return Ok(new
        {
            Message = "Order updated successfull
            Order = order
        });
    }
}
// ==============================================
// MINIMAL API WITH FILTERS (.NET 10)
// ==============================================
public static class OrderV2Endpoints
{
    public static void MapOrderV2Endpoints(this
```

```
        {
            var group = app.MapGroup("/api/v2/orders
                .AddEndpointFilter<ReactiveValidatic

            group.MapPost("/", (OrderDto order) =>
            {
                // Pure business logic
                return Results.Created($"/api/v2/ord
                {
                    Message = "Order created",
                    Order = order
                });
            })
            .WithName("CreateOrderV2")
            .WithOpenApi();

            group.MapGet("/", () =>
            {
                var orders = new[]
                {
                    new { Id = 1, Product = "Laptop"
                    new { Id = 2, Product = "Mouse",
                };

                return Results.Ok(orders);
            });
        }
    }
    // ==============================================
    // AUDIT CONSUMER SERVICE
    // ==============================================
    public class AuditConsumerService : BackgroundSe
    {
        private readonly ILogger<AuditConsumerServic
        public AuditConsumerService(ILogger<AuditCon
        {
            _logger = logger;
        }
        protected override Task ExecuteAsync(Cancell
        {
            return Task.Run(() =>
            {
                // Subscribe to audit events for rea
                AuditLogStream.AuditEvents.Subscribe
                {
                    // In production, this would wri
```

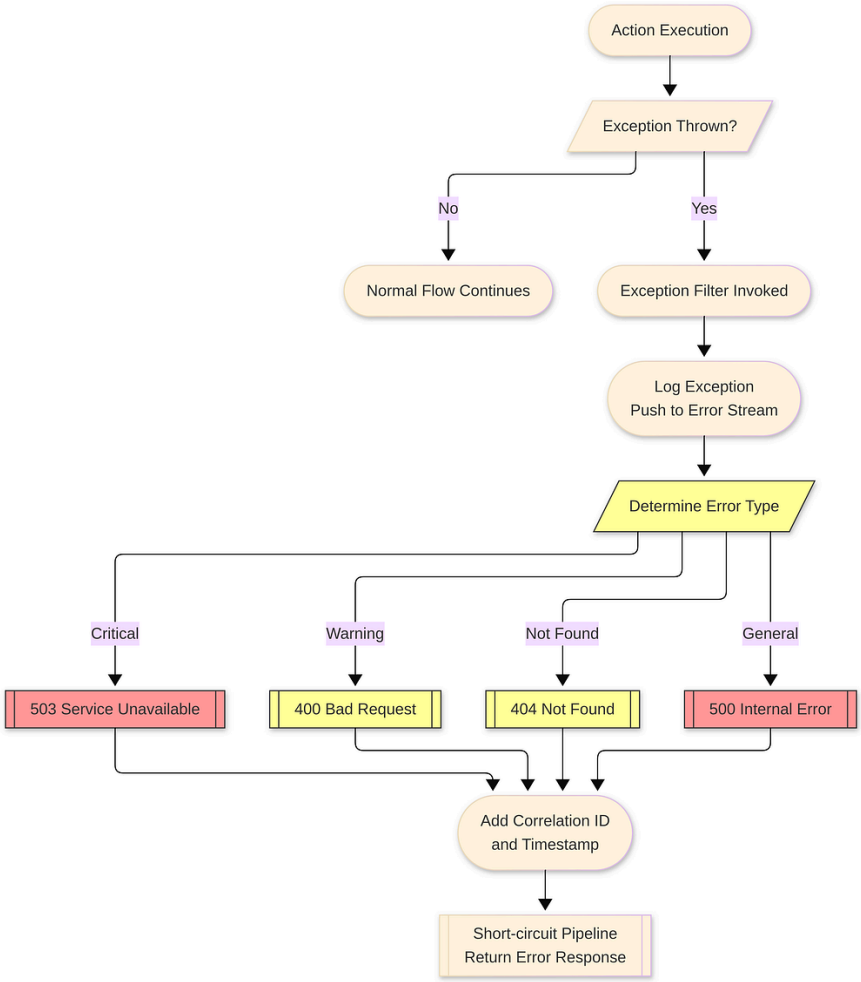
```
                _logger.LogInformation(
                    "AUDIT: [{Timestamp}] {User}
                    auditEvent.Timestamp,
                    auditEvent.User,
                    auditEvent.Action,
                    auditEvent.IpAddress,
                    auditEvent.CorrelationId
                );

                // Could also write to database,
                File.AppendAllText(
                    "audit.log",
                    $"{JsonSerializer.Serialize(
                );
            });

            // Monitor for suspicious activity
            AuditLogStream.AuditEvents
                .Buffer(TimeSpan.FromMinutes(5))
                .Subscribe(batch =>
                {
                    var suspicious = batch
                        .GroupBy(a => a.User)
                        .Where(g => g.Count() >
                        .Select(g => g.Key);

                    foreach (var user in suspici
                    {
                        _logger.LogWarning("SUSP
                            user, batch.Count(a
                    }
                });
        }, stoppingToken);
    }
}
```

. . .

4. Exception Filters: The Global Error Handlers

What They Do: Exception Filters handle unhandled exceptions that occur during action execution, model binding, or other filters. This is the ideal place to log errors and return standardized error responses to the client.

Exception Handling Flow



Exception Handling Flow [View Source]

The Four Benefits in Action:

- **Separation of Concerns:** Error handling is extracted from business logic
- **Cleaner Endpoints:** No try-catch blocks cluttering the code

- **Reusable Logic:** Same error handling across all endpoints
- **Consistent Errors:** Every exception produces a standardized response

Code Example: Reactive Error Handling with Notifications

```
using Microsoft.AspNetCore.Mvc.Filters;
using Microsoft.AspNetCore.Mvc;
using System.Reactive.Subjects;
using System.Reactive.Concurrency;

// ==============================================
// ERROR STREAM FOR REACTIVE PROCESSING
// ==============================================
public static class GlobalErrorStream
{
    // Hot observable for error events
    public static Subject<ErrorContext> Errors {

    // Derived streams for different severity le
    public static IObservable<ErrorContext> Crit
        Errors.Where(e => e.Exception is Critica

    public static IObservable<ErrorContext> Warn
        Errors.Where(e => e.Exception is Warning
}
public record ErrorContext(
    Exception Exception,
    string Path,
    string Method,
    string User,
    DateTime Timestamp,
    string CorrelationId,
    object? RequestData = null);
// Custom exception types for better categorizat
public class CriticalException : Exception
{
    public CriticalException(string message) : b
    public CriticalException(string message, Exc
}
public class WarningException : Exception
```

```
{
    public WarningException(string message) : ba
}
// ==============================================
// EXCEPTION FILTER - REUSABLE ERROR HANDLER
// ==============================================
public class ReactiveGlobalExceptionFilter : IEx
{
    private readonly ILogger<ReactiveGlobalExcep
    private readonly IWebHostEnvironment _env;
    public ReactiveGlobalExceptionFilter(
        ILogger<ReactiveGlobalExceptionFilter> l
        IWebHostEnvironment env)
    {
        _logger = logger;
        _env = env;
    }
    public void OnException(ExceptionContext con
    {
        // Push to reactive stream
        PushToErrorStream(context);

        // Create standardized response
        var response = CreateErrorResponse(conte

        // Set the result (short-circuit the pip
        context.Result = new ObjectResult(respon
        {
            StatusCode = GetStatusCode(context.E
        };

        context.ExceptionHandled = true;
    }
    public Task OnExceptionAsync(ExceptionContex
    {
        OnException(context);
        return Task.CompletedTask;
    }
    private void PushToErrorStream(ExceptionCont
    {
        var errorContext = new ErrorContext(
            Exception: context.Exception,
            Path: context.HttpContext.Request.Pa
            Method: context.HttpContext.Request.
            User: context.HttpContext.User.Ident
            Timestamp: DateTime.UtcNow,
```

```
                CorrelationId: context.HttpContext.I
                RequestData: GetRequestData(context.
            );

            GlobalErrorStream.Errors.OnNext(errorCon
        }
        private object CreateErrorResponse(Exception
        {
            var correlationId = context.HttpContext.
                                context.HttpContext.T

            // Base response (safe for production)
            var response = new
            {
                Error = "An error occurred while pro
                Type = GetErrorType(context.Exceptic
                CorrelationId = correlationId,
                Timestamp = DateTime.UtcNow
            };

            // Include details in development
            if (_env.IsDevelopment())
            {
                return new
                {
                    Error = context.Exception.Messag
                    Type = context.Exception.GetType
                    StackTrace = context.Exception.S
                    CorrelationId = correlationId,
                    Timestamp = DateTime.UtcNow,
                    InnerException = context.Excepti
                };
            }

            return response;
        }
        private int GetStatusCode(Exception ex) => e
        {
            UnauthorizedAccessException => 401,
            ArgumentException => 400,
            KeyNotFoundException => 404,
            CriticalException => 503, // Service Una
            _ => 500
        };
        private string GetErrorType(Exception ex) =>
        {
```

```
            CriticalException => "Critical",
            WarningException => "Warning",
            _ => "General"
        };
        private object? GetRequestData(HttpRequest r
        {
            try
            {
                // Safely capture request data (avoi
                return new
                {
                    Query = request.QueryString.ToSt
                    Headers = request.Headers
                        .Where(h => !h.Key.Equals("A
                        .ToDictionary(h => h.Key, h
                    ContentType = request.ContentTyp
                };
            }
            catch
            {
                return null; // Don't let request re
            }
        }
    }
    // ==============================================
    // CONTROLLER - COMPLETELY FREE OF ERROR HANDLIN
    // ==============================================
    [ApiController]
    [Route("api/[controller]")]
    [ServiceFilter(typeof(ReactiveGlobalExceptionFil
    public class PaymentsController : ControllerBase
    {
        private static readonly Dictionary<int, deci

        [HttpPost("process/{orderId}")]
        public IActionResult ProcessPayment(int orde
        {
            // NO TRY-CATCH BLOCKS!
            // NO ERROR HANDLING CODE!
            // Just business logic - if it throws, f

            if (orderId <= 0)
            {
                throw new ArgumentException("Invalid
            }
```

```
            if (request.Amount <= 0)
            {
                throw new ArgumentException("Payment
            }

            // Simulate database operation that migh
            if (!SimulatePaymentProcessing(orderId,
            {
                throw new CriticalException("Payment
                    new Exception("Connection timeou
            }

            _payments[orderId] = request.Amount;

            return Ok(new
            {
                Message = "Payment processed success
                TransactionId = Guid.NewGuid(),
                Amount = request.Amount
            });
        }

        [HttpGet("risky")]
        public IActionResult RiskyOperation()
        {
            // This will throw and be caught by the
            throw new WarningException("This operati
        }

        private bool SimulatePaymentProcessing(int o
        {
            // Simulate random failures
            return Random.Shared.Next(0, 10) > 2; //
        }
    }
    public record PaymentRequest(decimal Amount, str
    // ==============================================
    // MINIMAL API WITH EXCEPTION FILTER (.NET 10)
    // ==============================================
    var builder = WebApplication.CreateBuilder();
    builder.Services.AddScoped<ReactiveGlobalExcepti
    var app = builder.Build();
    // Apply filter to a group
    var paymentGroup = app.MapGroup("/api/payments")
        .AddEndpointFilter<ReactiveGlobalExceptionFi
    paymentGroup.MapPost("/process/{orderId}", (int
```

```
{
    // No error handling - let filter manage it
    if (request.Amount > 10000)
    {
        throw new CriticalException("Large payme
    }

    return Results.Ok(new { Status = "Processed"
});
paymentGroup.MapGet("/status/{transactionId}", (
{
    // This might throw if transaction not found
    if (transactionId.Length < 10)
    {
        throw new KeyNotFoundException($"Transac
    }

    return Results.Ok(new { Status = "Completed"
});
// ==============================================
// ERROR MONITORING SERVICE
// ==============================================
public class ErrorMonitoringService : Backgroun
{
    private readonly ILogger<ErrorMonitoringServ
    public ErrorMonitoringService(ILogger<ErrorM
    {
        _logger = logger;
    }
    protected override Task ExecuteAsync(Cancell
    {
        return Task.Run(() =>
        {
            // Real-time error logging
            GlobalErrorStream.Errors.Subscribe(e
            {
                _logger.LogError(
                    error.Exception,
                    "ERROR: [{Timestamp}] {Path}
                    error.Timestamp,
                    error.Path,
                    error.Exception.Message,
                    error.CorrelationId
                );
            });
```

```
            // Alert on critical errors
            GlobalErrorStream.CriticalErrors
                .SubscribeOn(TaskPoolScheduler.D
                .Subscribe(criticalError =>
                {
                    _logger.LogCritical(
                        criticalError.Exception,
                        "CRITICAL: Immediate att
                        criticalError.Path,
                        criticalError.Exception.
                    );

                    // Send SMS/PagerDuty/Teams
                    SendAlert(criticalError);
                });

            // Error rate monitoring
            GlobalErrorStream.Errors
                .Buffer(TimeSpan.FromMinutes(5))
                .Subscribe(batch =>
                {
                    if (batch.Count > 50)
                    {
                        _logger.LogWarning("High
                    }

                    // Group by error type
                    var byType = batch.GroupBy(e
                                      .Select(g

                    foreach (var item in byType)
                    {
                        _logger.LogInformation("
                    }
                });
        }, stoppingToken);
    }

    private void SendAlert(ErrorContext critical
    {
        // Implementation for SMS, PagerDuty, Te
        Console.WriteLine($"🚨 ALERT: Critical e
```

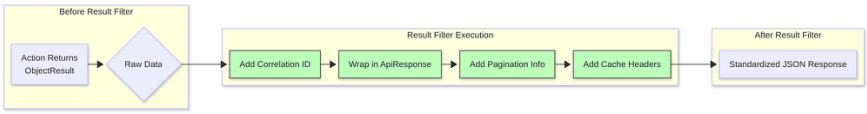
```
        }
    }
```

. . .

5. Result Filters: The Response Formatters

What They Do: Result Filters execute *before* and *after* the action result is processed. This is useful when you need to add headers to the response, modify the response body, or compress the output.

Response Transformation Flow



Response Transformation Flow [View Source]

The Four Benefits in Action:

- **Separation of Concerns:** Response formatting is separated from business logic
- **Cleaner Endpoints:** Controllers return simple objects, not formatted responses
- **Reusable Logic:** Same response envelope across all endpoints
- **Consistent API:** Every response follows the same structure

Code Example: Standardized API Responses with Caching

```
using Microsoft.AspNetCore.Mvc.Filters;
using Microsoft.AspNetCore.Mvc;
using System.Text.Json;
using Microsoft.Net.Http.Headers;
```

```
// ==============================================
// STANDARD API RESPONSE ENVELOPE
// ==============================================
public class ApiResponse<T>
{
    public T? Data { get; set; }
    public string? Message { get; set; }
    public bool Success { get; set; }
    public DateTime Timestamp { get; set; }
    public string? CorrelationId { get; set; }
    public PaginationInfo? Pagination { get; set
    public Dictionary<string, string[]>? Errors
}
public class PaginationInfo
{
    public int Page { get; set; }
    public int PageSize { get; set; }
    public int TotalCount { get; set; }
    public int TotalPages { get; set; }
    public bool HasPrevious => Page > 1;
    public bool HasNext => Page < TotalPages;
}
// ==============================================
// RESULT FILTER FOR STANDARDIZED RESPONSES
// ==============================================
public class StandardizedResponseFilter : IResul
{
    private readonly ILogger<StandardizedRespons
    public StandardizedResponseFilter(ILogger<St
    {
        _logger = logger;
    }
    public void OnResultExecuting(ResultExecutin
    {
        // Skip for certain result types
        if (context.Result is EmptyResult ||
            context.Result is FileResult ||
            context.Result is PhysicalFileResult
        {
            return;
        }
        // Handle ObjectResult (most common)
        if (context.Result is ObjectResult objec
        {
            var correlationId = context.HttpCont
```

```
                                    context.HttpConte

            // Create standardized envelope
            var envelope = CreateEnvelope(object

            // Replace the result
            objectResult.Value = envelope;

            // Ensure status code is preserved
            if (objectResult.StatusCode == null)
            {
                objectResult.StatusCode = contex
                    200 : context.HttpContext.Re
            }
        }

        // Add standard headers
        context.HttpContext.Response.Headers["X-
        context.HttpContext.Response.Headers["X-
            context.HttpContext.Items["Correlati
    }
    public void OnResultExecuted(ResultExecutedC
    {
        // Log response size for monitoring
        if (context.HttpContext.Response.Content
        {
            _logger.LogDebug("Response size: {Si
        }
    }
    private ApiResponse<object> CreateEnvelope(c
    {
        var response = new ApiResponse<object>
        {
            Data = originalValue,
            Success = context.HttpContext.Respon
            Timestamp = DateTime.UtcNow,
            CorrelationId = correlationId
        };

        // Add message based on status code
        response.Message = context.HttpContext.R
        {
            200 => "Request completed successful
            201 => "Resource created successfull
            400 => "Bad request - please check y
            401 => "Authentication required",
```

```
                403 => "You don't have permission to
                404 => "Resource not found",
                500 => "Internal server error",
                _ => null
            };

            // Add pagination info if present in Htt
            if (context.HttpContext.Items.TryGetValu
            {
                response.Pagination = pagination as
            }

            return response;
        }
    }
    // ==============================================
    // CACHING RESULT FILTER
    // ==============================================
    public class CacheControlFilter : IResultFilter
    {
        private readonly int _durationSeconds;
        public CacheControlFilter(int durationSecond
        {
            _durationSeconds = durationSeconds;
        }
        public void OnResultExecuting(ResultExecutin
        {
            // Skip for non-GET requests
            if (!string.Equals(context.HttpContext.R
            {
                return;
            }
            var response = context.HttpContext.Respo

            // Add cache headers
            response.Headers[HeaderNames.CacheContro
            response.Headers[HeaderNames.Expires] =
            response.Headers[HeaderNames.Vary] = "Ac
        }
        public void OnResultExecuted(ResultExecutedC
        {
            // Nothing to do after execution
        }
    }
    // ==============================================
    // CONTROLLER - CLEAN AND FOCUSED
```

```
// ==============================================
[ApiController]
[Route("api/[controller]")]
[ServiceFilter(typeof(StandardizedResponseFilter
public class CatalogController : ControllerBase
{
    private static readonly List<Product> _produ
    {
        new Product { Id = 1, Name = "Laptop", P
        new Product { Id = 2, Name = "Mouse", Pr
        new Product { Id = 3, Name = "Keyboard",
        new Product { Id = 4, Name = "Monitor",
    };
    [HttpGet]
    [ServiceFilter(typeof(CacheControlFilter))]
    public IActionResult GetProducts([FromQuery]
    {
        // Just return the data - filter will wr
        var pagedProducts = _products
            .Skip((page - 1) * pageSize)
            .Take(pageSize)
            .ToList();

        // Add pagination info to context (will
        HttpContext.Items["Pagination"] = new Pa
        {
            Page = page,
            PageSize = pageSize,
            TotalCount = _products.Count,
            TotalPages = (int)Math.Ceiling(_prod
        };

        return Ok(pagedProducts); // Returns Api
    }
    [HttpGet("{id}")]
    public IActionResult GetProduct(int id)
    {
        var product = _products.FirstOrDefault(p

        if (product == null)
        {
            return NotFound(); // Filter will wr
        }

        return Ok(product); // Returns ApiRespon
    }
```

```
        [HttpPost]
        public IActionResult CreateProduct([FromBody
        {
            // Simulate creation
            product.Id = _products.Max(p => p.Id) +
            _products.Add(product);

            return CreatedAtAction(nameof(GetProduct
            // Returns ApiResponse<Product> with 201
        }
    }
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Em
        public decimal Price { get; set; }
        public string Category { get; set; } = strin
    }
    // ==============================================
    // MINIMAL API WITH RESPONSE FILTERS (.NET 10)
    // ==============================================
    var builder = WebApplication.CreateBuilder();
    builder.Services.AddScoped<StandardizedResponseF
    builder.Services.AddScoped<CacheControlFilter>()
    var app = builder.Build();
    // Apply filters to groups
    var apiV1 = app.MapGroup("/api/v1/catalog")
        .AddEndpointFilter<StandardizedResponseFilte
        .AddEndpointFilter<CacheControlFilter>();
    apiV1.MapGet("/products", (int? page, int? pageS
    {
        page ??= 1;
        pageSize ??= 10;

        var products = new[]
        {
            new { Id = 1, Name = "Tablet", Price = 5
            new { Id = 2, Name = "Case", Price = 20
        };

        // Return raw data - filter handles wrapping
        return Results.Ok(products);
    });
    apiV1.MapGet("/products/{id}", (int id) =>
    {
        if (id == 999)
```

```
        {
            return Results.NotFound(); // Filter wil
        }

        var product = new { Id = id, Name = $"Produc
        return Results.Ok(product);
    });
    apiV1.MapPost("/products", (Product product) =>
    {
        // Return 201 with location header
        return Results.Created($"/api/v1/catalog/pro
    });
    // ==============================================
    // EXAMPLE RESPONSES (What the client sees)
    // ==============================================
    /*
    GET /api/v1/catalog/products/1
    {
        "data": {
            "id": 1,
            "name": "Laptop",
            "price": 1200,
            "category": "Electronics"
        },
        "message": "Request completed successfully",
        "success": true,
        "timestamp": "2024-01-15T10:30:00Z",
        "correlationId": "abc-123-def",
        "pagination": null,
        "errors": null
    }
    GET /api/v1/catalog/products/999
    {
        "data": null,
        "message": "Resource not found",
        "success": false,
        "timestamp": "2024-01-15T10:30:00Z",
        "correlationId": "xyz-789-uvw",
        "pagination": null,
        "errors": null
    }
    Headers:
    X-API-Version: 1.0
    X-Correlation-ID: abc-123-def
    Cache-Control: public, max-age=60
    */
```

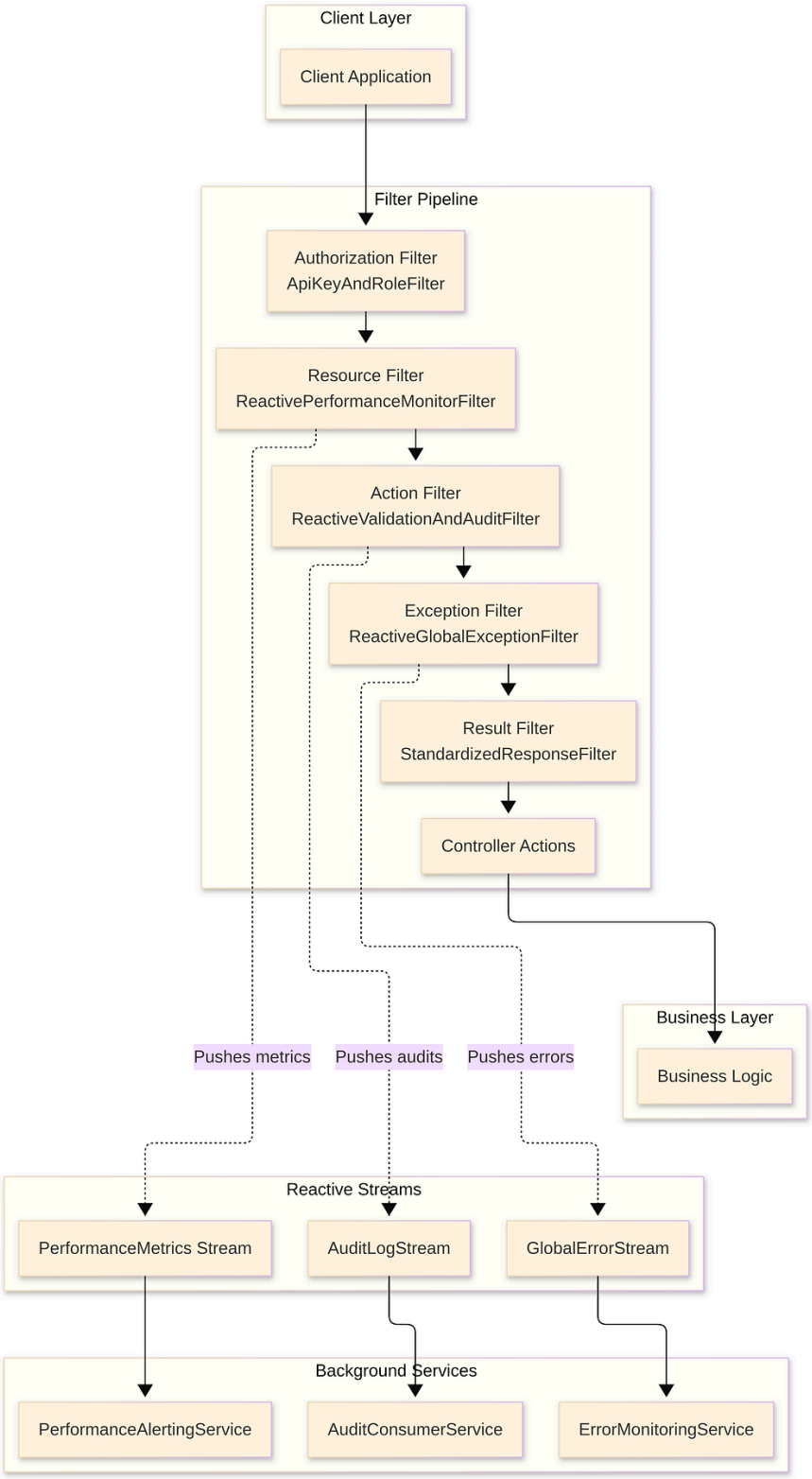
```
// ==============================================
// REACTIVE RESPONSE ANALYZER
// ==============================================
public class ResponseAnalyzerService : Backgroun
{
    private readonly ILogger<ResponseAnalyzerSer
    public ResponseAnalyzerService(ILogger<Respo
    {
        _logger = logger;
    }
    protected override Task ExecuteAsync(Cancell
    {
        // This would hook into response events
        // For now, it's a placeholder showing h

        return Task.CompletedTask;
    }
}
app.Run();
```

. . .

Putting It All Together: The Complete Picture

Now let's see how all these filters work together in a complete application:

Complete Filter Pipeline Architecture



Complete Filter Pipeline Architecture [View Source]

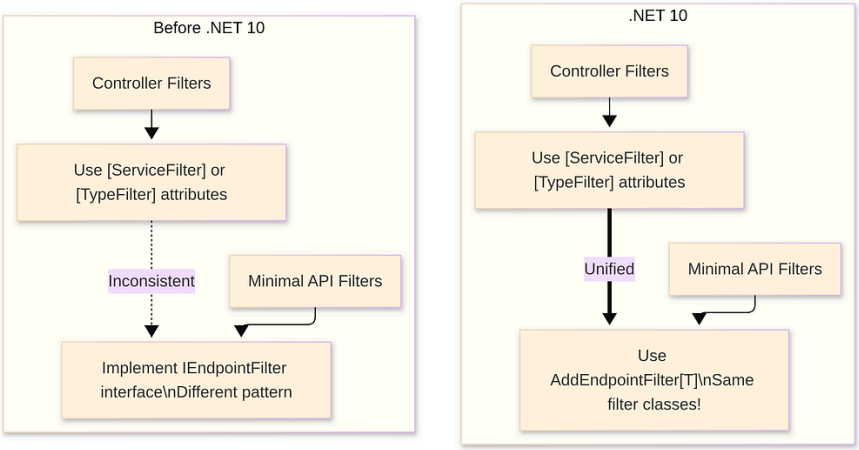
Complete Program.cs with All Filters

```
// ==============================================
// PROGRAM.CS - .NET 10 APPLICATION WITH ALL FIL
// ==============================================

var builder = WebApplication.CreateBuilder(args)
// Add services
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
// Register all filters
builder.Services.AddScoped<ApiKeyAndRoleFilter>(
builder.Services.AddScoped<ReactivePerformanceMo
builder.Services.AddScoped<ReactiveValidationAnd
builder.Services.AddScoped<ReactiveGlobalExcepti
builder.Services.AddScoped<StandardizedResponseF
builder.Services.AddScoped<CacheControlFilter>()
// Register background services
builder.Services.AddHostedService<PerformanceAle
builder.Services.AddHostedService<AuditConsumerS
builder.Services.AddHostedService<ErrorMonitorin
// Configure authentication
builder.Services.AddAuthentication(JwtBearerDefa
    .AddJwtBearer(options =>
    {
        options.Authority = "https://your-identi
        options.Audience = "api";
    });
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("Admin", policy => policy.
    options.AddPolicy("User", policy => policy.R
});
var app = builder.Build();
// Configure pipeline
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
app.UseHttpsRedirection();
// Important: Filters run in the middleware pipe
app.UseAuthentication();
app.UseAuthorization();
// Map controller routes (filters applied via at
```

```
app.MapControllers();
// Map minimal API routes with filters
app.MapGroup("/api/minimal/orders")
    .AddEndpointFilter<ApiKeyAndRoleFilter>()
    .AddEndpointFilter<ReactiveValidationAndAudi
    .AddEndpointFilter<StandardizedResponseFilte
    .MapOrderEndpoints(); // Extension method fr
app.MapGroup("/api/minimal/payments")
    .AddEndpointFilter<ReactiveGlobalExceptionFi
    .AddEndpointFilter<StandardizedResponseFilte
    .MapPaymentEndpoints();
app.Run();
// ==============================================
// SUMMARY: THE FOUR BENEFITS ACHIEVED
// ==============================================
/*
 * ✅ SEPARATION OF CONCERNS:
 *    - Authentication logic is in ApiKeyAndRole
 *    - Validation logic is in ReactiveValidatic
 *    - Error handling is in ReactiveGlobalExcep
 *    - Response formatting is in StandardizedRe
 *    - Performance monitoring is in ReactivePer
 *
 * ✅ CLEANER CONTROLLERS/ENDPOINTS:
 *    - Controllers contain ONLY business logic
 *    - No authentication checks cluttering the
 *    - No validation code in action methods
 *    - No try-catch blocks everywhere
 *    - No manual response formatting
 *
 * ✅ REUSABLE LOGIC ACROSS ENDPOINTS:
 *    - Same authentication filter protects all
 *    - Same validation logic applies to all POS
 *    - Same error handler catches exceptions ev
 *    - Same response formatter standardizes all
 *    - Same performance monitor tracks all requ
 *
 * ✅ CONSISTENT LOGGING AND ERROR HANDLING:
 *    - Every authenticated request is logged th
 *    - Every validation error has the same form
 *    - Every exception produces the same struct
 *    - Every response has the same envelope str
```

```
    *     - Every request is monitored with the same
    */
```

Filter Registration Comparison: .NET 10 vs Previous Versions



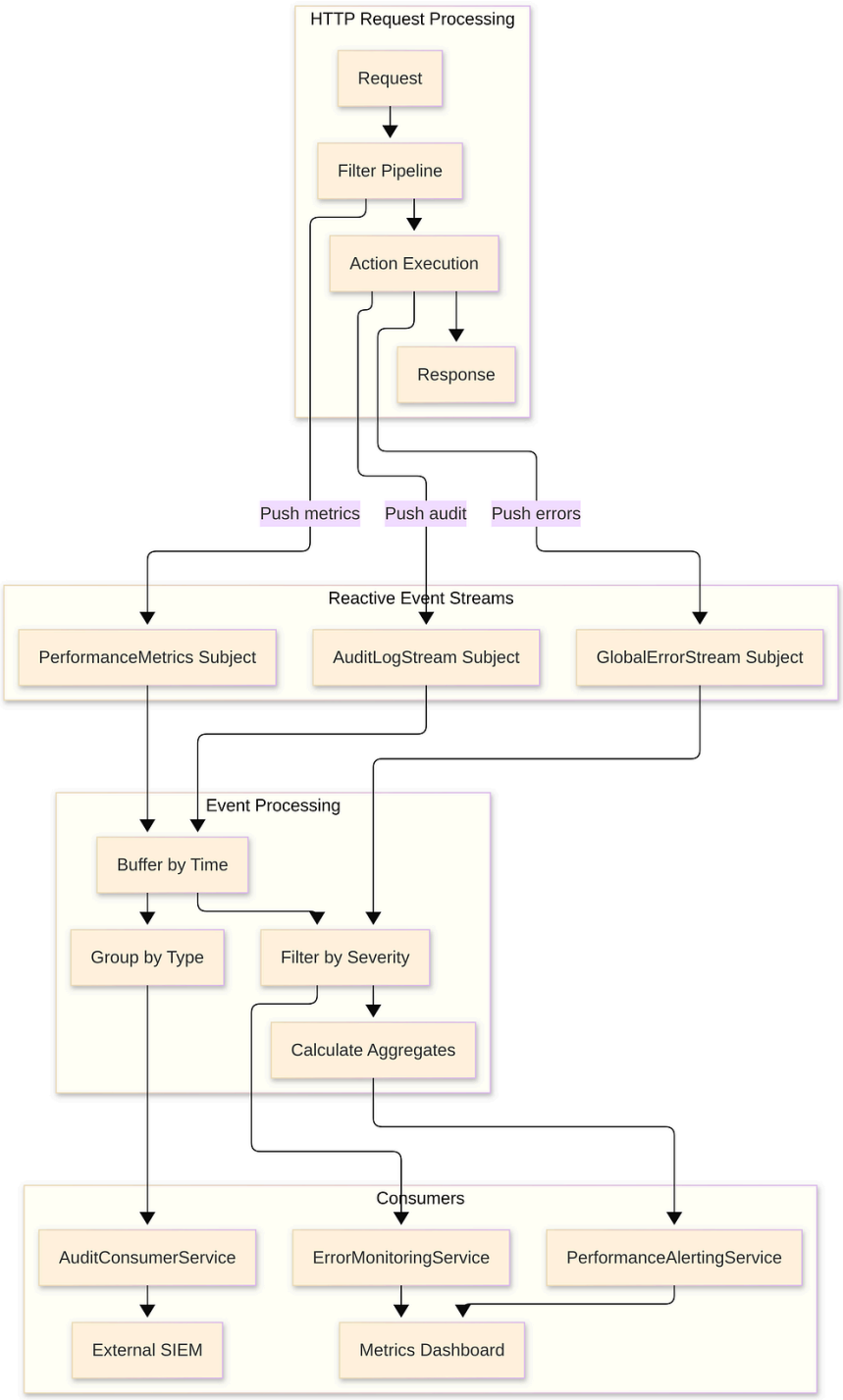
Filter Registration Comparison: .NET 10 vs Previous Versions [View Source]

What's Changed from Before .NET 10:

Feature	Before .NET 10	.NET 10
Minimal API Filters	`IEndpointFilter` interface (different pattern)	`AddEndpointFilter<T>` with same filters as controllers
Filter Registration	Separate patterns for controllers vs. minimal	Unified registration system
Async Support	Good, but inconsistent	Fully unified `IAsyncActionFilter` etc.
Dependency Injection	Required `ServiceFilterAttribute`	Works seamlessly in both styles
Reactive Integration	Manual implementation	Built-in support with improved Activity APIs

What's Changed from Before .NET 10 [View Source]

Reactive Streams Architecture



Reactive Streams Architecture [View Source]

. . .

Conclusion: The Power of a Structured, Reactive Pipeline

Are you using filters in your ASP.NET Core APIs? If not, you're missing out on one of the most powerful features of the framework. By leveraging the structured pipeline of Authorization, Resource, Action, Exception, and Result filters, you can achieve true separation of concerns.

Key Takeaways:

1. **Separation of Concerns**—Each filter handles one specific cross-cutting concern, leaving your business logic pure and focused
2. **Cleaner Controllers**—Your action methods contain only the code that makes your application unique
3. **Reusable Logic**—Write once, apply everywhere. The same authentication filter can protect both traditional controllers and modern minimal APIs
4. **Consistent Behavior**—Every endpoint behaves predictably with standardized responses, error formats, and logging

With .NET 10, this pattern is more robust and unified than ever. The gap between controller-based APIs and minimal APIs has closed, allowing you to use the same filter patterns regardless of which style you prefer.

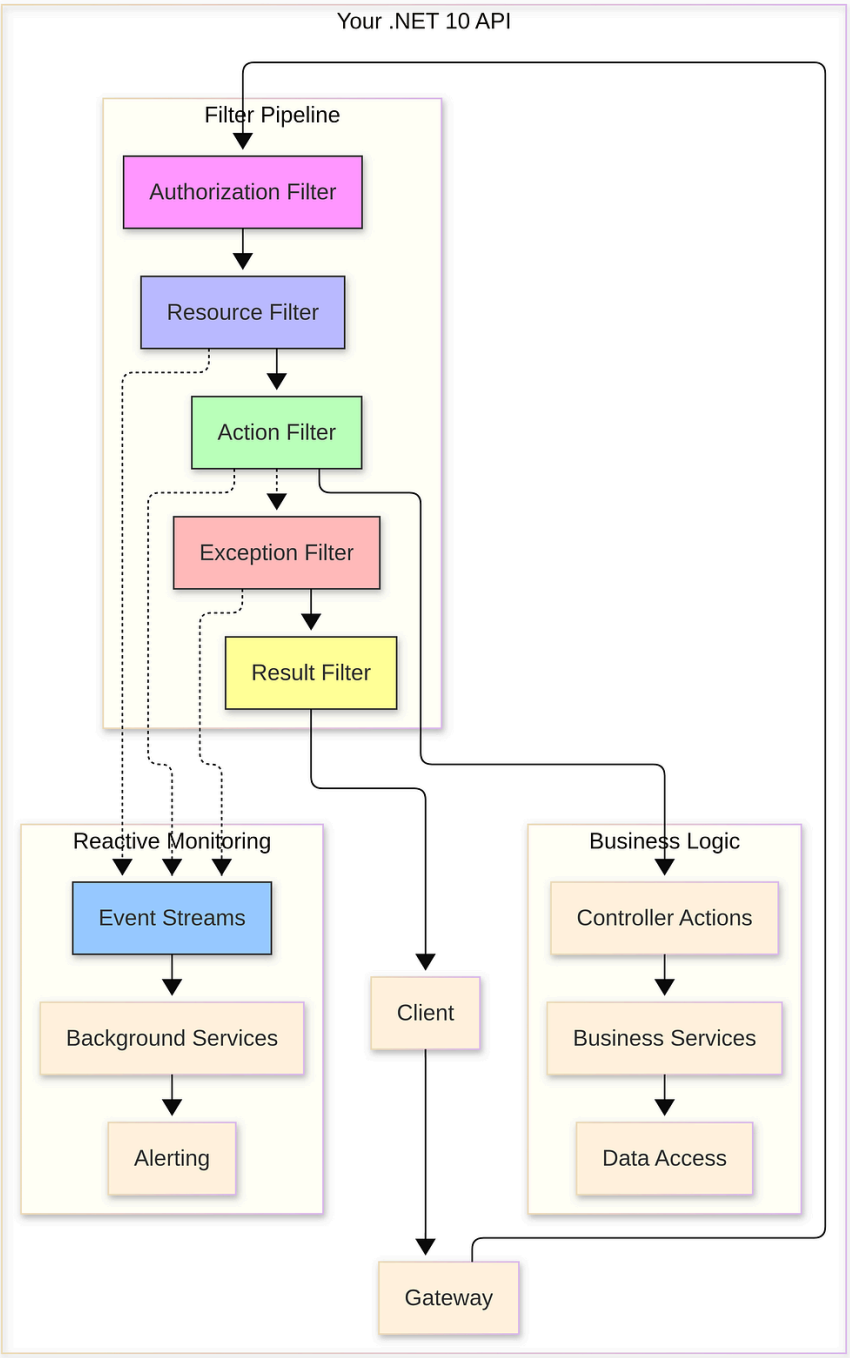
And by integrating Reactive Programming principles, you transform these interceptors from simple imperative blocks into the nervous system of your application, streaming events and decoupling your cross-cutting logic from your business logic.

Your Action Plan:

1. **Start Small**—Add one filter (like logging) to a single endpoint
2. **Expand Gradually**—Add validation filters to your POST endpoints
3. **Go Global**—Implement global exception handling and response formatting

4. **Go Reactive**—Use `System.Reactive` to process streams of events from your filters
5. **Monitor & Improve**—Use the data from your reactive streams to optimize performance

Final Architecture Overview



Final Architecture Overview [View Source]

Which filters are you most excited to implement in your next .NET 10 project? Let me know in the comments!

Follow me for more deep dives into ASP.NET Core, clean architecture, and reactive programming patterns.

. . .

💬 **Questions?** Drop a response—I read and reply to every comment.

📌 **Save this story** to your reading list—it helps other engineers discover it.

🔗 **Follow me** →

- **Medium**—mvineetsharma.medium.com
- LinkedIn—www.linkedin.com/in/vineet-sharma-architect
- *In-depth .NET, Node.js, Python, Cloud Architecture, and System Design. New articles weekly*

#dotnet #aspnetcore #webapi #softwarearchitecture #dotnet10 #reactiveprogramming #csharp #cleanarchitecture #programming #devcommunity